

No capítulo 6, Padrões de Projeto, debatidos pelo autor, são soluções aprovadas e testadas para problemas recorrentes no campo do desenvolvimento de software. Esses padrões são uma espécie de receita para ajudar estruturar seus desafios de design de uma forma mais eficiente e bem organizada. Dessa forma, os desenvolvedores economizam força evitando reinventar a roda, resolvendo problemas existentes.

Há três categorias principais: padrões criacionais para ajudar a criar objetos; padrões estruturais para organizar relacionamentos entre classes e objetos que formam grandes estruturas de software; padrões comportamentais para tratar da interação entre sistemas. O que se revelou interessante no ponto de vista do autor é a facilitação da comunicação sobre problemas de software dentro da equipe que usa uma abordagem de design orientada por padrões. Afinal, é muito mais fácil para os desenvolvedores compartilharem ideias usando o mesmo nome para um padrão do que dar uma definição para o que eles acham relevante.

No entanto, o autor adverte que não se pode fazer uso excessivo ou despropositado de padrões. Embora os padrões sejam muito valiosos como ferramenta, eles podem afetar negativamente a qualidade do código se não houver planejamento. Em resumo, a lição desse capítulo que eu acho que posso compartilhar é: Padrões de Projeto são sempre úteis, mas apenas se você souber usá-los vagarosamente. O excesso deles vai tornar seu código complicado, em vez de torná-lo simples.

Já o sétimo capítulo desse livro é focado em arquitetura de software, que é a organização estrutural de um sistema e das interações de suas diferentes partes. Isso define a base para o comportamento de um conjunto de elementos do software, garantindo que ele satisfaça tanto os requisitos funcionais quanto não funcionais, como desempenho e segurança e manutenibilidade. Para isso, o autor descreve alguns estilos arquiteturais. Alguns deles incluem a arquitetura monolítica em que o sistema é desenvolvido como um bloco único e integrado quanto à arquitetura de microserviços que dividem o sistema em pequenos serviços independentes, que podem ser desenvolvidos e implantados de forma independente.

A arquitetura monolítica é mais simples quando você começa, mas fica muito difícil de escalar à medida o sistema cresce. Microserviços fornecem mais flexibilidade e escalabilidade, mas introduzem a complexidade de integrar e gerenciar muitas peças menores. Eventos são outra arquitetura discutida neste capítulo. Os componentes se comunicam por eventos assíncronos. Isso é ótimo para sistemas distribuídos e altamente escaláveis, mas dificulta a consistência de dados e a orquestração de serviços.

Eu concordo com a maneira como o autor aborda a importância de escolher uma arquitetura que permita que o sistema evolua à medida que cresce. Muitos problemas de manutenção e escalabilidade podem ser evitados com um planejamento arquitetônico cuidadoso. Além disso, o autor exclui as fraquezas de cada abordagem, sem sinalizar uma abordagem como a única correta.

Os Capítulos 6 e 7 discutem dois aspectos vitais relacionados ao desenvolvimento de software: padrões de design e arquitetura de software. Os padrões de design são

soluções eficientes para problemas comuns, combinando abordagens colaborativas para sua implementação. Por outro lado, a arquitetura é responsável pelo sistema, pela maneira como ele é estruturado, pela maneira como resolve os problemas da vida real, como escalabilidade e manutenção. Eles têm algo em comum, precauções que precisam ser tomadas, o uso excessivo de padrões levará a um código complexo, enquanto a arquitetura sem um plano criará a fricção do sistema à medida que você o desenvolve.

A ideia-chave é tomar decisões ponderadas que atendam ao objetivo do seu projeto e de sua equipe, manter a simplicidade e a funcionalidade. O que o livro propõe é que os desenvolvedores, ao melhorar essas práticas, possam criar sistemas prontos para qualquer desafio futuro que a tecnologia possa lançar.